

Slide-by-Slide Speaker Notes & Anticipated Q&A

Keep this on your laptop – do not show on screen

Slide 3: Architecture Diagram

Say this (30 sec)

“Everything flows from config files. Observations go into the agent, the agent outputs two continuous actions, the controller checks for captures, the reward engine assigns credit, and the logger records everything to CSV. No hardcoded values anywhere.”

Likely questions

Q: Why is the controller a god-class?

A: It manages episode lifecycle, capture detection, curriculum advancement, and reward triggering. In Unity ML-Agents, this pattern is standard – the environment controller orchestrates all agents. We separated rewards and sensing into their own modules to keep it maintainable.

Open: Environment/PursuitEvasionEnvController.cs

Q: How do agents communicate?

A: They do not communicate directly. Each agent sees only its own observations. Coordination emerges from shared training (parameter sharing) and tactical reward shaping, not from message passing.

Q: Why YAML configs instead of Inspector values?

A: YAML lets us version-control experiments, diff configurations across runs, and swap reward weights without opening Unity. It also makes the setup reproducible – anyone can clone the repo and run the same experiment.

Slide 4: Agent Hierarchy

Say this (20 sec)

“BaseAgent handles movement, observation collection, and ML-Agents integration. SentinelAgent adds capture-radius logic. RunnerAgent adds escape state management. All 3 Sentinels share one neural network. Both Runners share another.”

Likely questions

Q: What does parameter sharing actually mean?

A: Literally the same weights. Sentinel A, B, and C call the same neural network. Each gets different observations because they are at different positions, so they output different actions. It is like 3 employees with the same training manual but looking at different situations.

Q: Can agents specialize with parameter sharing?

A: Not structurally – they cannot learn “I am agent A, I should always go left.” But they can learn role-conditioned behavior: “if I am closest to the Runner, chase; if I am far, cut off routes.” The observation determines the behavior, not the agent ID.

Q: Why not give each agent its own network?

A: 3x less data per network update. With 3 Sentinels sharing, every experience from any Sentinel trains the shared brain. This is especially valuable early in training when data is scarce.

Open: Agents/BaseAgent.cs – show CollectObservations

Slide 5: Observation Pipeline

Say this (30 sec)

“117 floats for Sentinels, 129 for Runners. Not pixels – numbers. Position, velocity, heading, raycasts, memory of last-known target position, and a summary of the 2 nearest opponents. Runners get 2 extra rays because they need 360-

degree threat detection.”

Likely questions

Q: Why not use camera observations?

A: Camera observations require CNNs, are much slower to train, and are harder to interpret. Vector observations train faster, are fully interpretable (we know exactly what each float means), and are standard practice for state-based RL.

Q: What is the last-known-position memory?

A: When a target leaves line-of-sight, we store its last XYZ position and time since disappearance. The agent keeps “remembering” where the target was. This decays as time passes. Without it, agents forget targets the moment they go behind a wall.

Open: Sensors/LastKnownPositionMemory.cs

Q: Why 14 rays for Sentinels vs 16 for Runners?

A: Runners are prey – they need wider coverage to detect threats from all directions. Sentinels are predators – they focus on forward pursuit. More rays = more observation floats = slightly larger input but better spatial awareness.

Q: What do the 6 floats per ray mean?

A: Float 1 = normalized hit distance (0 to 1). Floats 2–6 = one-hot encoding: wall, exit, teammate, opponent, other. So the agent knows what it hit and how far away it is.

Open: Sensors/RaySensorBuilder.cs

Q: What is the opponent summary?

A: For the 2 nearest opponents: relative XZ position, distance, LOS flag (can I see them?), and threat pressure (how fast are they approaching?). This gives a structured view of the most dangerous enemies without needing to process all rays.

Open: Sensors/ObservationAssembler.cs around line 139

Slide 6: Reward Engine

Say this (30 sec)

“Terminal rewards are ± 1.0 for win/loss. Everything else is tiny – thousandths. This is intentional: shaping rewards guide behavior but the agent always prioritizes winning. All weights come from YAML, so we can tweak without recompiling.”

Likely questions

Q: Why are shaping rewards so small?

A: If chase progress ($+0.003/\text{step}$) were too large, agents might learn to chase forever instead of actually capturing. The small magnitude ensures terminal reward (± 1.0) always dominates the cumulative shaping.

Q: How does the TrapEventDetector work?

A: Every step, it checks geometric conditions: are 2 Sentinels on opposite sides of a Runner (pincer)? Is a Sentinel blocking both ends of a corridor? Is a Sentinel near an exit while a Runner approaches? These are spatial checks on agent positions, not learned behaviors.

Open: Rewards/TrapEventDetector.cs

Q: Could agents exploit the shaping rewards?

A: Theoretically yes. An agent could orbit a Runner to farm chase-progress rewards without capturing. We added orbit-stall penalties (-0.0025) and wall-loop penalties to counter this. The reward audit CSV logs every reward event so we can check for exploitation.

Q: Why separate policies for Sentinels and Runners?

A: They have different objectives. Sentinels want to capture.

Runners want to escape or survive. The reward functions are inverted. Separate policy classes keep the logic clean.

Open: Rewards/SentinelRewardPolicy.cs VS RunnerRewardPolicy.cs

Slide 7: Training Pipeline

Say this (30 sec)

“We start easy and get harder. Static maze first, then random spawns, then slow wall shifts, then fast wall shifts. Agents must hit both a minimum episode count and a win-rate threshold before advancing. This is not arbitrary – we tried training on dynamic mazes directly and it failed.”

Likely questions

Q: What happens if you skip the curriculum?

A: Agents fail to learn basic pursuit. With walls shifting every 8 seconds from the start, the environment is too non-stationary for agents to discover that chasing leads to reward. The curriculum lets them learn chase → generalize → adapt.

Q: Why these specific thresholds (0.62, 0.55, 0.50, 0.45)?

A: They decrease because each stage is harder. Stage 1 (static, fixed) is easiest, so we require 62% before advancing. Stage 4 (dynamic, 8s) is hardest, so 45% is sufficient. The thresholds were tuned empirically.

Q: Why 500K max steps?

A: Compute budget constraint. Reward curves showed convergence around 300K–400K steps. 500K gives a safety margin.

Open: configs/curriculum_configs/curriculum_3v2_full_v1.yaml

Q: What is the time horizon (64)?

A: How many decision steps the agent collects before sending a batch to PPO for update. Short horizon = faster updates, more bias. Long horizon = slower updates, less bias. 64 is a standard ML-Agents default.

Slide 8: Logging & Results

Say this (20 sec)

“We log everything: episode outcomes, per-step agent positions, every reward event, and tactical events. Results are averaged across 3 seeds on both seen and unseen layouts. Win rates are stable. Capture timing shifts by 7.5 seconds on unseen mazes.”

Likely questions

Q: Why is capture faster on unseen mazes?

A: Our hypothesis is that unseen procedural layouts produce more constrained corridors, which limit Runner escape routes. Sentinels catch Runners faster because there are fewer paths to flee. The policy transfers, but it is topology-sensitive.

Q: Is 3 seeds enough?

A: For a course project, yes. Results are consistent across all three (no outlier seed). For a publication, we would use 5–10 seeds with confidence intervals.

Q: Why didn't you measure coordination directly?

A: “We deliberately chose outcome-level metrics because they are objective and reproducible. We have the step-level logs to compute pincer rate, coverage spread, and corridor-block frequency. We list this as future work because we wanted to report only metrics we can fully defend.”

HIGH PRIORITY – asked in another group's viva!

After Slide 8: Live Demo (no slide – just do it)

Say this

“Let me show you the trained agents running in the dynamic maze. No Python is running – this is pure ONNX inference inside Unity.”

During demo, point at

- Sentinels spreading out (cluster penalty)
- Runner heading toward exit (exit reward)
- Wall shift happening (dynamic topology)
- Capture event (terminal reward trigger)

If demo fails

“Let me show the pre-recorded video instead.” Open Google Drive link from the paper footnote.

Likely questions during demo

Q: Why are Sentinels blue and not red?

A: Legacy material naming. The file is called Sentinel_Red.mat but the actual color is blue. Same with Runner – Runner_Blue.mat is actually pink. We kept the names because Unity uses GUID references. Renaming would break prefab links.

Q: Are they actually coordinating or just chasing?

A: “Watch this – two Sentinels approaching from opposite sides. That pattern is consistent with pincer behavior. But we are honest in the paper: we report outcome metrics, not coordination proof. The behavior is suggestive, not conclusive.”

Q: What happens when walls shift?

A: Agents re-plan. They do not get stuck because the observation updates every step with new ray distances. The wall shift changes what the rays hit, which changes the action output.

Slide 9: Summary

Say this (20 sec)

“This is a modular, config-driven, fully-logged system. It is reproducible – the repo is public, the configs are versioned, the models are included. Our honest limitations: no MA-POCA comparison, no ablation, no coordination metrics. Those are concrete next steps, not vague future work.”

Final questions

Q: What would you do with more time?

A: Three things. (1) Run MA-POCA and compare with PPO on the same curriculum. (2) Ablation: turn off memory and measure win rate delta. Turn off shaping and measure. (3) Compute coordination metrics from the step logs we already have.

Q: What was the hardest part?

A: Getting the curriculum right. Training directly on dynamic mazes with 8-second wall shifts produced zero learning. We had to step back, train on static mazes first, and gradually increase complexity. That is when we understood why curriculum learning matters – not from the paper, from the failure.

Q: Did you use any LLM tools?

A: “We used LLM tools for drafting and code scaffolding. All technical decisions, hyperparameter choices, and experimental results are our own work. The code and trained models are in the public repo.”